

 Marwadi University	Marwadi University Faculty of Technology Department of Information and Communication Technology	
Subject: Digital VLSI Testing (01CT0629)	Aim: To test memory cells using March test patterns.	
Experiment No: 13	Date: 10-04-2026	Enrolment No: 92301733056

AIM: To test memory cells using March test patterns.

IDE: EDA Playground

Theory:

Memory Testing ensures that SRAM or DRAM cells can store and retrieve data correctly and without interference from neighbouring cells. One of the most effective algorithms for memory testing is the March Test.

What is a March Test?

A March Test consists of a sequence of memory operations (Read/Write) applied in a specific address order (ascending or descending). The operations detect a wide range of memory faults like:

- Stuck-at faults (cell always reads 0 or 1),
- Transition faults (cell can't switch states),
- Coupling faults (operation on one cell affects another),
- Address decoder faults.

Example: March C- Test

This is a commonly used march sequence:

1. $\uparrow(w0)$ — write 0 in ascending order
2. $\uparrow(r0, w1)$ — read 0 and write 1 in ascending order
3. $\downarrow(r1, w0)$ — read 1 and write 0 in descending order

sequence:

- Verifies data is correctly written and read,
- Detects if writing to one location affects another (coupling),
- Helps catch transition/stuck-at/address faults.

Testing memory structures is crucial for ensuring reliable digital system behavior. One common and efficient approach for testing RAM is the March Test, a widely used memory fault detection algorithm. This experiment demonstrates the design and implementation of a 4x4-bit SRAM module and tests it using March Test sequences to identify stuck-at and coupling faults. The design and simulation are performed using System Verilog on EDA Playground.

Memory elements like SRAM are integral to digital systems. Testing such memories during manufacturing is essential to detect permanent and intermittent faults. Traditional approaches involve complex external testers. However, using March Test sequences in a self-contained simulation provides

 Marwadi University	Marwadi University Faculty of Technology Department of Information and Communication Technology	
Subject: Digital VLSI Testing (01CT0629)	Aim: To test memory cells using March test patterns.	
Experiment No: 13	Date: 10-04-2026	Enrolment No: 92301733056

a simpler and cost-effective solution for identifying common memory faults like stuck-at, transition, and coupling faults.

MARCH TEST ALGORITHM

A March Test is a sequence of read/write operations applied in ascending and descending address order. For this experiment, we use a simplified March C- sequence:

1. **March 0 (Ascending):** Write 0 → Read 0
2. **March 1 (Ascending):** Write 1 → Read 1
3. **March -1 (Descending):** Write 0 → Read 0

This test helps identify stuck-at-0, stuck-at-1, and transition faults.

DESIGN AND IMPLEMENTATION:

A. SRAM Design

A 4x4-bit memory module is created using a 2-bit address input, read/write control (we), and synchronous clocked write operations. Asynchronous reads are handled using combinational logic.

B. March Testbench

The testbench applies the March sequence by writing and reading each address. Delays and edge-sensitive triggers ensure proper synchronization.

VERILOG CODE:

A. SRAM Module

```
// Code your design here
module sram_4x4 (
    input clk,    input we,
    // write enable  input [1:0]
    addr,        // 2-bit address
    input [3:0] din,    // 4-bit
    input data    output reg [3:0]
    dout // 4-bit output data
);

reg [3:0] mem [0:3]; // memory array (4 locations)

// Write operation (synchronous)
always @(posedge clk) begin
```

 Marwadi University	Marwadi University Faculty of Technology Department of Information and Communication Technology	
Subject: Digital VLSI Testing (01CT0629)	Aim: To test memory cells using March test patterns.	
Experiment No: 13	Date: 10-04-2026	Enrolment No: 92301733056

```

        if (we)
        mem[addr] <=
        din;
        end

        // Read operation (asynchronous)
        always @(*) begin
            if (!we)
            dout =
            mem[addr];
            else
            dout =
            4'b0000;    end

    endmodule

```

B. March Test

Testbench module tb_sram;

```

    reg clk;    reg we;    reg [1:0] addr;    reg [3:0] din;
    wire [3:0] dout;

```

```

    integer i;

```

```

    // Instantiate SRAM

```

```

sram_4x4 uut (
    .clk(clk),
    .we(we),
    .addr(addr),
    .din(din),
    .dout(dout)
);

```

```

    // Clock generation

```

```

    always #5 clk = ~clk;

```



Marwadi University
Faculty of Technology
Department of Information and Communication Technology

**Subject: Digital VLSI
Testing (01CT0629)**

Aim: To test memory cells using March test patterns.

Experiment No: 13

Date: 10-04-2026

Enrolment No: 92301733056

```
// Write
Task task
write_mem;
input [1:0]
a; input
[3:0] data;
begin
    @(posedge clk);
    we
= 1;
addr = a;
din = data;
end
endtask

// Read +
Check Task
task read_check;
input [1:0] a;
input [3:0]
expected;
begin
    @(posedge clk);
    we
= 0;
addr = a;
    #1;

    if (dout !== expected)
        $display("ERROR at addr %0d: Expected=%b, Got=%b", a,
expected, dout);
    else
        $display("PASS at addr %0d: Data=%b", a,
dout);
    end
```



Marwadi University
Faculty of Technology
Department of Information and Communication Technology

**Subject: Digital VLSI
Testing (01CT0629)**

Aim: To test memory cells using March test patterns.

Experiment No: 13

Date: 10-04-2026

Enrolment No: 92301733056

endtask

initial begin

// Dump waveform

\$dumpfile("sram.vcd");

\$dumpvars(0, tb_sram);

//

Initializ

e

clk = 0;

we = 0;

addr =

0;

din = 0;

//

=====

// March 0: ↑ (w0, r0)

//

=====

\$display("---- March 0 -

---"); for (i = 0; i < 4; i

= i + 1) begin

write_mem(i, 4'b0000);

read_check(i, 4'b0000);

end

//

=====

// March 1: ↑ (w1, r1)

//

=====



Marwadi University
Faculty of Technology
Department of Information and Communication Technology

**Subject: Digital VLSI
Testing (01CT0629)**

Aim: To test memory cells using March test patterns.

Experiment No: 13

Date: 10-04-2026

Enrolment No: 92301733056

```
        $display("---- March 1 -  
---");    for (i = 0; i < 4; i  
= i + 1) begin  
write_mem(i, 4'b1111);  
read_check(i, 4'b1111);  
        end  
  
        //  
=====
```

```
//  March -1: ↓ (w0, r0)  
        //  
=====
```

```
        $display("---- March -1 -  
---");    for (i = 3; i >= 0; i  
= i - 1) begin  
write_mem(i, 4'b0000);  
read_check(i, 4'b0000);  
        end  
  
        $display("March Test  
Completed");  
        $finish;  
    end  
  
endmodule
```

SIMULATION RESULTS:

 Marwadi University	Marwadi University Faculty of Technology Department of Information and Communication Technology	
Subject: Digital VLSI Testing (01CT0629)	Aim: To test memory cells using March test patterns.	
Experiment No: 13	Date: 10-04-2026	Enrolment No: 92301733056



CONCLUSION:

The March Test algorithm successfully verifies the correct operation of SRAM by detecting faults like stuck-at, transition, and coupling faults.

The implemented 4×4 SRAM with testbench ensures reliable read/write functionality using structured test sequences.

Thus, March testing provides an efficient and cost-effective method for memory fault detection in digital systems.

 Marwadi University	Marwadi University Faculty of Technology Department of Information and Communication Technology	
Subject: Digital VLSI Testing (01CT0629)	Aim: To test memory cells using March test patterns.	
Experiment No: 13	Date: 10-04-2026	Enrolment No: 92301733056

Post Lab Exercise:

1. What is the purpose of using the March C- algorithm in RAM testing?

The **March C- algorithm** is used to **detect memory faults efficiently** in RAM.

Purpose:

- Detect **stuck-at faults** (cell always 0 or 1)
- Detect **transition faults** (cannot change state)
- Detect **coupling faults** (one cell affects another)
- Verify **read/write correctness** **Why March C-?**
- Simple and fast
- Covers most common memory faults
- Uses both **ascending and descending operations** for better fault coverage

2. How does a write enable (we) signal control read and write operations in a RAM module?

The **write enable (we)** signal decides whether the memory performs a **read or write operation**.

- **we = 1 → Write Operation** ○ Data is written into the memory at the given address (on clock edge)
- **we = 0 → Read Operation** ○ Data is read from memory and sent to output

Summary:

- we = 1 → Store data
- we = 0 → Retrieve data

3. Why are ascending and descending address orders used in March test patterns?, Using both address directions helps detect **more types of faults**.

Ascending order (↑):

- Tests memory sequentially from low to high address
- Helps detect basic faults **Descending order (↓):**
- Tests memory from high to low address

 Marwadi University	Marwadi University Faculty of Technology Department of Information and Communication Technology	
Subject: Digital VLSI Testing (01CT0629)	Aim: To test memory cells using March test patterns.	
Experiment No: 13	Date: 10-04-2026	Enrolment No: 92301733056

- Helps detect **coupling and address decoder faults** **Reason:**
- Some faults only appear when accessing memory in a **specific order**
- Combining both ensures **complete coverage**

4. How can the testbench detect a stuck-at fault (SA0 or SA1) in memory cells?

A **stuck-at fault** means:

- SA0 → cell always reads **0**
- SA1 → cell always reads **1** **Detection method:**
 1. Write a value (e.g., 1)
 2. Read back the value
 3. Compare expected vs actual.

Example:

- Write 1Read → if output is 0 → **SA0 fault detected**

5. What is the role of \$dumpfile and \$dumpvars in waveform generation during simulation?

These are used in **Verilog simulation** to generate waveform files.

- **\$dumpfile("file.vcd")** ○ Creates a VCD (Value Change Dump) file ○ Stores signal activity
- **\$dumpvars**
 - Records changes of variables/signals during simulation.

Purpose:

- Helps visualise signals in tools like GTKWave
- Useful for debugging and verification